ENSF 380

TOPIC 10: STRINGS PART I

Unicode

Standard character encoding using hexadecimal

- e.g. Ux0041 = Latin A

Defines characters from multiple languages around the world as code points

Compatible with ASCII

Hello = U+0048 U+0065 U+006C U+006C U+006F

Uses an encoding system to store the code points

- e.g. UTF-8 stores Unicode code points using 8 bit bytes
- Hello = 48 65 6C 6C 6F

Important for internationalization of text

Java Keywords

Pre-defined and reserved words

- e.g. class, integer, for, etc.

void: specifies that a method does not have a return value

static:

- Can be applied to variables, methods, blocks, and nested classes
- Member belongs to its own type- one instance that is shared
- Memory efficient

final:

- Can be applied to variables, methods, and classes
- Prevents variable values from being changed
- Prevents methods from being overridden
- Prevents classes from being extended

String Class

Java provides built-in support for strings

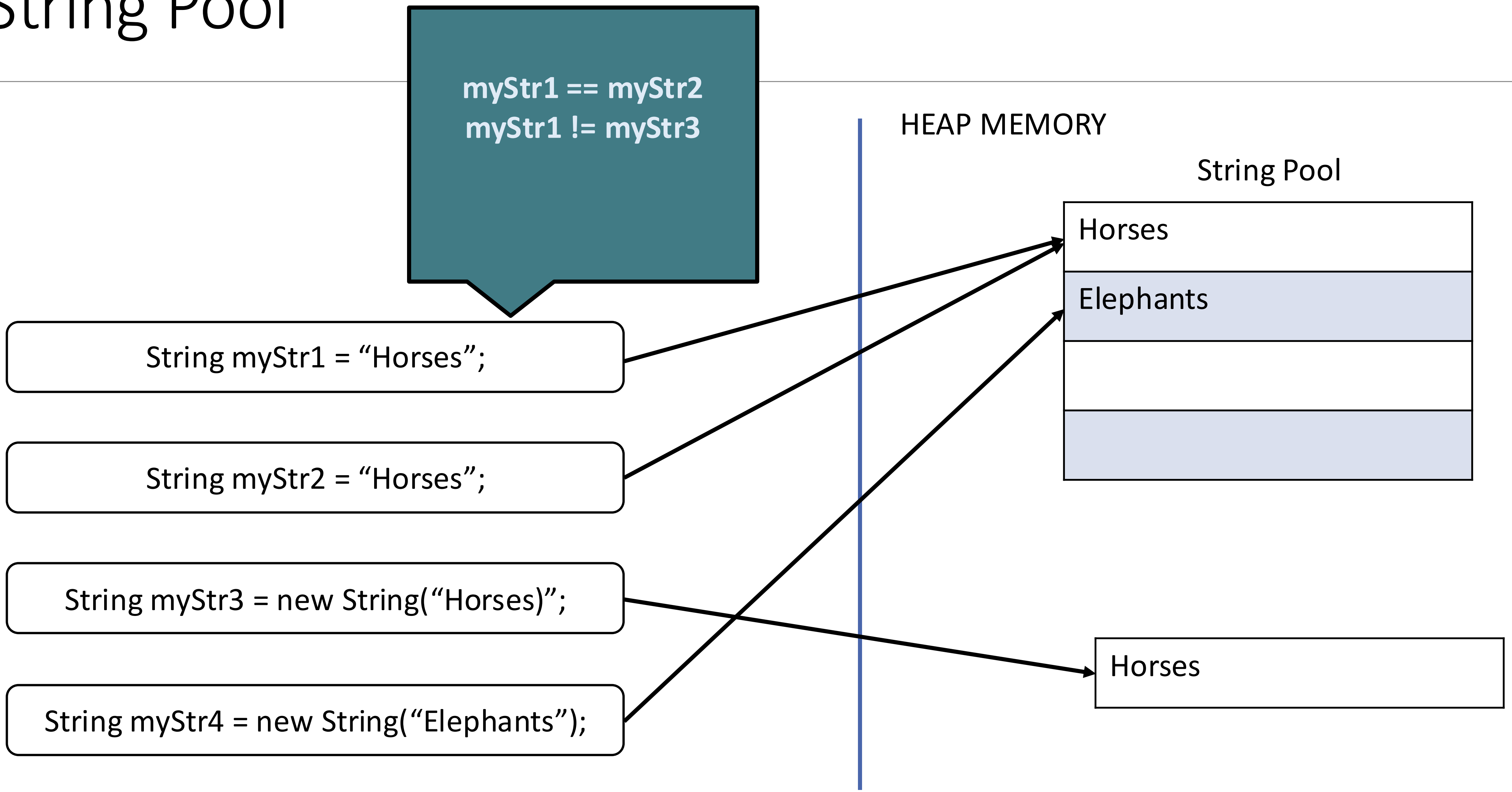
Non-primitive reference type

Final (immutable) = fixed-length and read-only, cannot be changed

A string object is implicitly created when a string literal is found in source code (e.g. “a string”)

String Pool contains all literal String values

String Pool


mySt1 == myStr2
myStr1 != myStr3
HEAP MEMORY
String Pool
Horses
Elephants
String myStr1 = “Horses”;
String myStr2 = “Horses”;
String myStr3 = new String(“Horses)”;
String myStr4 = new String(“Elephants”);
Horses

String Class Constructors

Several constructors available:

- String exampleString = new String(); //creates an empty String object
- String exampleString = new String(“ABCDE”);
- String exampleString = new String(String otherExample);
- String exampleString = new String(charArray);
- String exampleString = new String(charArray, 5, 2); //copies a subset of a character array
- String exampleString = new String(byteArray);
- String exampleString = new String(byteArray, 4, 4); //copies a subset of a byte array

Creating Strings

01_CreatingStrings

…

```
public class JavaStrings {

    public static void main(String[] args) {

        String animalFact1 = "Horses are mammals.";
        String animalFact2 = new String("Elephants are mammals.");
        String animalFact3 = new String (animalFact1);

        System.out.println(animalFact1);
        System.out.println(animalFact2);
        System.out.println(animalFact3);
    }
}
```

Creating Strings

01_CreatingStrings

...

```
public class JavaStrings {

    public static void main(String[] args) {

        String animalFact1 = "Horses are mammals.";
        String animalFact2 = new String("Elephants are mammals.");
        String animalFact3 = new String (animalFact1);

        System.out.println(animalFact1);
        System.out.println(animalFact2);
        System.out.println(animalFact3);
    }
}
```

```
Horses are mammals.
Elephants are mammals.
Horses are mammals.
```

String Class Methods

String class includes many pre-defined methods

- myStr.length(); //returns length
- myStr.charAt(int index); //returns at value at specified index
- myStr.getChars(); //returns the entire set of characters in the String
- myStr.getChars(int srcBegin, int srcEnd, charArray[] dst, int dstBegin); //copies specified characters of myStr to charArray
- myStr.indexOf(char ch); //returns index of specified character value
- myStr.lastIndexOf(char ch); //returns largest index of specified character value
- myStr.compareTo(String myStr2); //uses lexicographical comparison
- myStr.equals (String myStr2); //uses lexicographical comparison
- myStr.substring(int beginIndex, int endIndex); //returns a subset between two specified indices
- myStr.substring(int startingIndex); //returns a subset starting at the specified index to the end
- myStr.startWith(“all”); //returns true if exampleString starts with “all”
- myStr.endWith(“est”); //returns true if exampleString ends with “est”

String Class Methods (cont.)

String class includes many pre-defined methods

- myStr.concat(myStr2); //concatenates myStr and myStr2
- myStr.replace('a', 'A'); //replace every occurrence of one character with another such as 'a' with 'A'
- myStr.toLowerCase(); //converts every character to lower case
- myStr.toUpperCase(); //converts every character to upper case
- myStr.trim(); //trims leading and trailing white spaces
- … and more!

Finding String Length

02_StringLength

...

```
public class JavaStrings {

    public static void main(String[] args) {

        String animalFact1 = "Horses are mammals.";
        String animalFact2 = new String("Elephants are mammals.");
        String animalFact3 = new String (animalFact1);

        System.out.println(animalFact1.length());  // Length = 19
        System.out.println(animalFact2.length());  // Length = 22
        System.out.println(animalFact3.length());  // Length = 19
    }
}
```

Exercise 10.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

String Concatenation

03_Concatenation

…

```
public class JavaStrings {

    public static void main(String[] args) {

        String animalFact1 = "Horses are mammals.";
        String animalFact2 = new String("Elephants are mammals.");
        String animalFact3 = new String (animalFact1);

        System.out.println("Fact 1: " + animalFact1 + " Fact 2: " +
          animalFact2);

        String animalFact4 = animalFact2 + animalFact3;
        System.out.println(animalFact4);
    }
}
```

String Concatenation

03_Concatenation

…

```
public class JavaStrings {

    public static void main(String[] args) {

        String animalFact1 = "Horses are mammals.";
        String animalFact2 = new String("Elephants are mammals.");
        String animalFact3 = new String (animalFact1);

        System.out.println("Fact 1: " + animalFact1 + " Fact 2: " +
          animalFact2);

        String animalFact4 = animalFact2 + animalFact3;
        System.out.println(animalFact4);
    }
}
```

```
Fact 1: Horses are mammals. Fact 2: Elephants are mammals.
Elephants are mammals.Horses are mammals.
```

String Casting

Various ways to cast to and from Strings

Most classes provide a toString() method for printing objects

Cast a primitive data type to String:

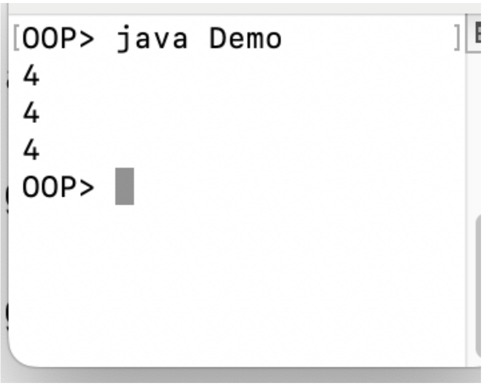
- String.valueOf();

Cast a String to a primitive data type:

- Integer.parseInt (String myStr)
- new Float(String myStr).floatValue()
- new Double(String myStr).doubleValue()

```
String myStr = "1";
int number = Integer.parseInt(myStr) + 3;
```

```
System.out.println(number);
System.out.println(String.valueOf(number));
System.out.println(Integer.toString(number));
```



Converting to a String

04_CastingToString

...

```
public class JavaStrings {

    public static void main(String[] args) {

        Double speedRecord = 70.76;
        System.out.println("The fastest record for a horse is " + String.valueOf(speedRecord)
            + " km/h.");
        System.out.println("The fastest record for a horse is " + Double.toString(speedRecord)
            + " km/h.");
        System.out.println("The fastest record for a horse is " + speedRecord + " km/h.");
    }
}
```

Converting to a String

04_CastingToString

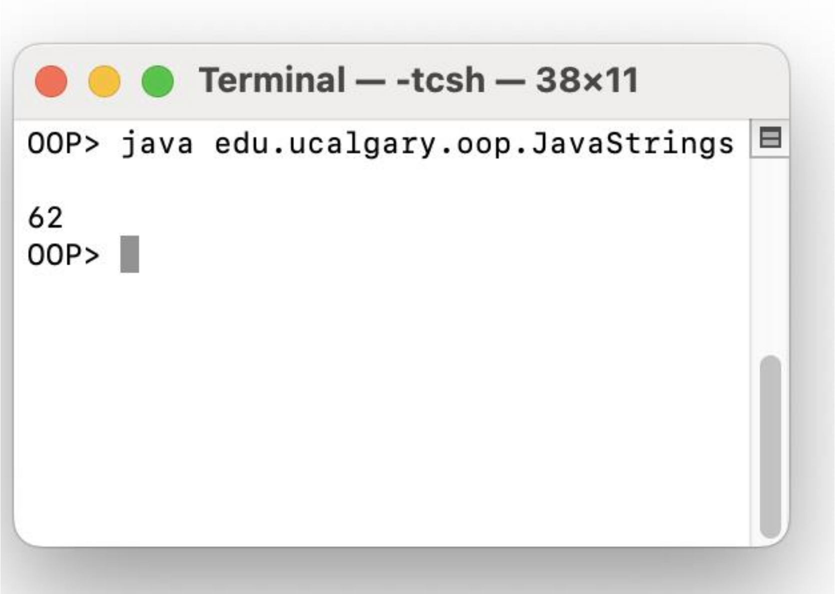
```
public class JavaStrings {

    public static void main(String[] args) {

        Double speedRecord = 70.76;
        System.out.println("The fastest record for a horse is " + String.valueOf(speedRecord)
            + " km/h.");
        System.out.println("The fastest record for a horse is " + Double.toString(speedRecord)
            + " km/h.");
        System.out.println("The fastest record for a horse is " + speedRecord + " km/h.");
    }
}
```

```
The fastest record for a horse is 70.76 km/h.
The fastest record for a horse is 70.76 km/h.
The fastest record for a horse is 70.76 km/h.
```

Converting to a String

05_CastingToString

```
package edu.ucalgary.oop;

public class JavaStrings {
    private static int firstNumber = 6;
    private static int secondNumber = 2;

    public String toString() {
        String one = Integer.toString(firstNumber) ;
        String two = Integer.toString(secondNumber);
        return one + two;
    }

    public static void main(String[] args) {
        var tmp = new JavaStrings();
        System.out.println(tmp);
    }
}
```



Converting to a String

05_CastingToString

```
package edu.ucalgary.oop;

public class JavaStrings {
    private static int firstNumber = 6;
    private static int secondNumber = 2;
/*
    public String toString() {
        String one = Integer.toString(firstNumber) ;
        String two = Integer.toString(secondNumber);
        return one + two;
    }
*/
    public static void main(String[] args) {
        var tmp = new JavaStrings();
        System.out.println(tmp);
    }
}
```

Terminal — -tcsh — 38×11

```
OOP> java edu.ucalgary.oop.JavaStrings

edu.ucalgary.oop.JavaStrings@515f550a
OOP>
```

Converting from a String

06_CastingFromString

…

```
public class JavaStrings {

  public static void main(String[] args) {

  String elephantWeight = "6000";
  String babyElephantWeight = "100";

  int sumElephants = Integer.parseInt(elephantWeight) + Integer.parseInt(babyElephantWeight);
  System.out.println("Adult and baby weigh " + sumElephants + " kg.");
  sumElephants = Integer.valueOf(elephantWeight) + Integer.valueOf(babyElephantWeight);
  System.out.println("Adult and baby weigh " + sumElephants + " kg.");

  }
}
```

Converting from a String

06_CastingFromString

…

```java
public class JavaStrings {

  public static void main(String[] args) {

  String elephantWeight = "6000";
  String babyElephantWeight = "100";

  int sumElephants = Integer.parseInt(elephantWeight) + Integer.parseInt(babyElephantWeight);
  System.out.println("Adult and baby weigh " + sumElephants + " kg.");
  sumElephants = Integer.valueOf(elephantWeight) + Integer.valueOf(babyElephantWeight);
  System.out.println("Adult and baby weigh " + sumElephants + " kg.");

  }
}
```

```
Adult and baby weigh 6100 kg.
Adult and baby weigh 6100 kg.
```

Creating Substrings

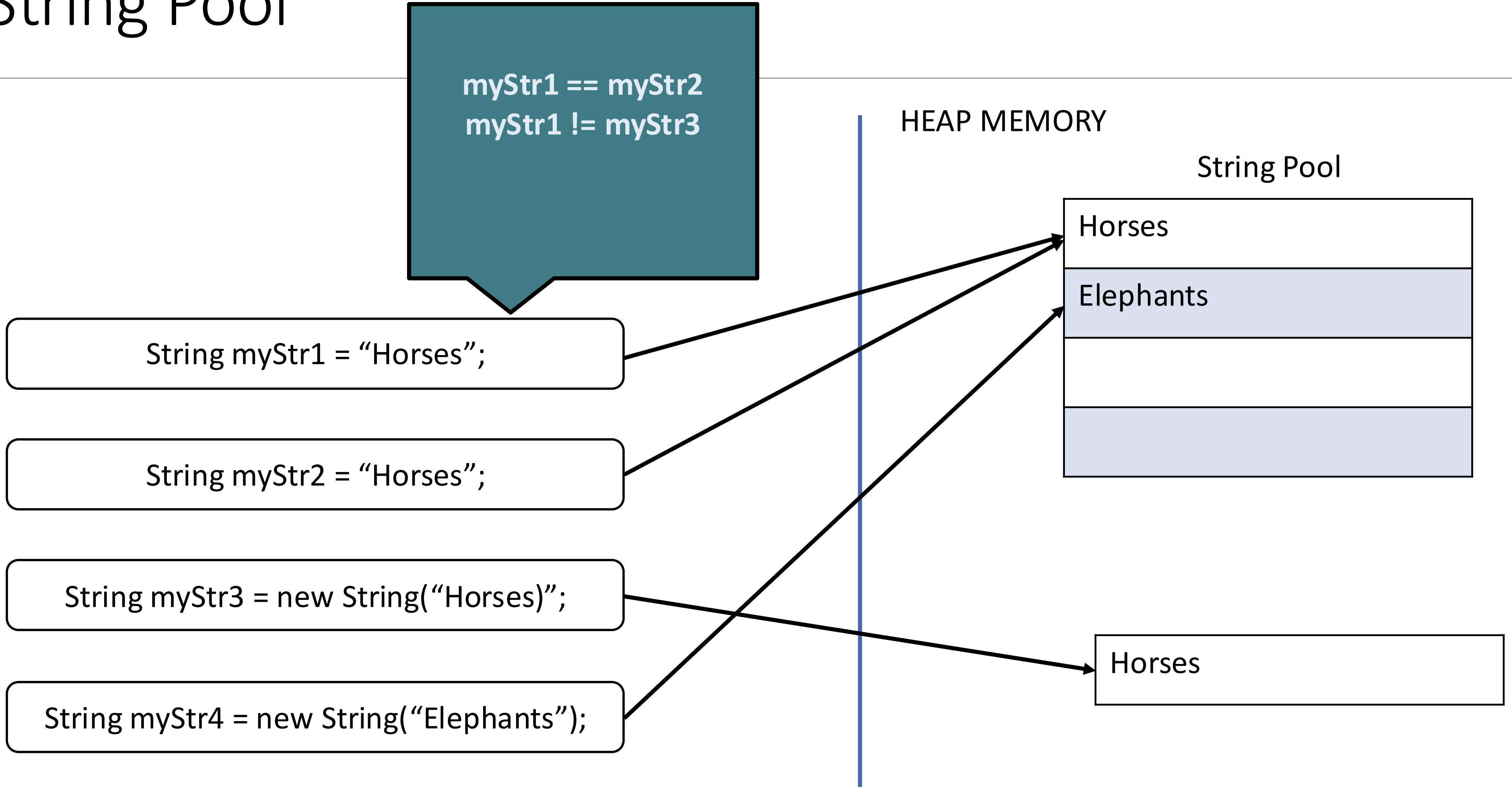
07_Substrings

...

```
public class JavaStrings {

    public static void main(String[] args) {

        String animalFact1 = "Horses are mammals.";

        System.out.println(animalFact1.substring(0,6));
        System.out.println(animalFact1.substring(6));
    }
}
```

```
Horses
 are mammals.
```

String Pool


myStr1 == myStr2
myStr1 != myStr3
String myStr1 = “Horses”;
String myStr2 = “Horses”;
String myStr3 = new String(“Horses)”;
String myStr4 = new String(“Elephants”);
HEAP MEMORY
String Pool
Horses
Elephants
Horses

Comparing Strings

08_CompareStrings

...

```
public class JavaStrings {

    public static void main(String[] args) {

        String animalFact1 = "Horses are mammals.";
        String animalFact2 = "Horses are mammals.";
        String animalFact3 = new String("Horses are mammals.");
        String animalFact4 = new String ("Elephants are mammals.");

        System.out.println(animalFact1 == animalFact2);
        System.out.println(animalFact1 == animalFact3);
        System.out.println(animalFact1.equals(animalFact2));
        System.out.println(animalFact1.equals(animalFact3));
        System.out.println(animalFact1.equals(animalFact4));
        System.out.println(animalFact1.compareTo(animalFact2));
        System.out.println(animalFact1.compareTo(animalFact4));

    }
}
```

Comparing Strings

08_CompareStrings

...

```
public class JavaStrings {

    public static void main(String[] args) {

        String animalFact1 = "Horses are mammals.";
        String animalFact2 = "Horses are mammals.";
        String animalFact3 = new String("Horses are mammals.");
        String animalFact4 = new String ("Elephants are mammals.");

        System.out.println(animalFact1 == animalFact2);
        System.out.println(animalFact1 == animalFact3);
        System.out.println(animalFact1.equals(animalFact2));
        System.out.println(animalFact1.equals(animalFact3));
        System.out.println(animalFact1.equals(animalFact4));
        System.out.println(animalFact1.compareTo(animalFact2));
        System.out.println(animalFact1.compareTo(animalFact4));

    }
}
```

```
true
false
true
true
false
0
3
```

Exercise 10.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

StringBuilder Class

Objects that allow text to be modified (mutable)

Variable-length sequence of characters with a designated capacity

Use String unless StringBuilder offers an advantage

- e.g. concatenating several strings together

Methods for appending, inserting, and deleting

Several constructors available:

- StringBuilder myBuilder1 = new StringBuilder(); //creates empty StringBuilder with capacity of 16
- StringBuilder myBuilder2 = new StringBuilder("ABCDE"); //contains specified chars plus 16 empty elements
- StringBuilder myBuilder3 = new StringBuilder(50); //creates empty StringBuilder with specified capacity
- StringBuilder myBuilder4 = new StringBuilder(myString); // contains specified string plus 16 empty elements

StringBuilder Class

09_StringBuilder

Use append() to add characters at the end of the sequence

Use insert() to add characters at a specified point

…

```
public class JavaStrings {

    public static void main(String[] args) {

        StringBuilder animalFact1 = new StringBuilder("Horses are mammals");
        animalFact1.append(" and so are elephants.");

        System.out.println(animalFact1);

        StringBuilder animalFact2 = new StringBuilder("Horses are mammals.");
        animalFact2.insert(7,"and elephants ");

        System.out.println(animalFact2);
    }
}
```

StringBuilder Class

09_StringBuilder

Use append() to add characters at the end of the sequence

Use insert() to add characters at a specified point

...

```
public class JavaStrings {

    public static void main(String[] args) {

        StringBuilder animalFact1 = new StringBuilder("Horses are mammals");
        animalFact1.append(" and so are elephants.");

        System.out.println(animalFact1);

        StringBuilder animalFact2 = new StringBuilder("Horses are mammals.");
        animalFact2.insert(7,"and elephants ");

        System.out.println(animalFact2);
    }
}
```

```
Horses are mammals and so are elephants.
Horses and elephants are mammals.
```

Summary: Knowledge

Unicode and important keywords

Basics of the String class

String Pool and memory allocation of Strings

String constructors and methods

Converting to and from Strings

Partitioning a string

StringBuilder constructors and methods

Summary: Skills

Instantiate and manipulate strings

Find the length of a string

Concatenate and cast with strings

Trim and replace letters in a string

Create a substring

Modify a string using StringBuilder

Additional Information

Related reading

- Volume 1, Chapter 3.6: Strings